Semantic Segmentation Pipeline

AI Training & Inference Documentation

DeepLabV3 / LRASPP MobileNetV3 • PyTorch • 10 Classes • 512×512 Input

Overview

This project implements a full semantic segmentation pipeline using PyTorch and torchvision. The model is trained to classify every pixel in an image into one of 10 nature/environment categories — Trees, Bushes, Grass, Flowers, Rocks, Sky, and more — producing detailed colorized segmentation maps with element labels and coverage statistics.

Model	LRASPP MobileNetV3-Large (falls back to DeepLabV3 ResNet-50)
Classes	10 nature/environment categories
Framework	PyTorch + torchvision + OpenCV + NumPy + Matplotlib
Output	Side-by-side original + segmentation + labeled result + legend panel

Requirements

System Requirements

- Python 3.8 or higher
- CUDA-compatible GPU (recommended) or CPU
- Minimum 4 GB RAM (8 GB recommended for training)

Install Dependencies

Run the following command to install all required packages:

```
pip install torch torchvision opencv-python numpy matplotlib Pillow
```

Dataset Folder Structure

Organize your dataset exactly as shown below. Mask files must share the same filename as their corresponding image.

```
project/
├── segmentation_pipeline.py
├── dataset/
│   ├── trainImages/
│   │   ├── images/          ← training photos (.jpg / .png)
│   │   └── masks/           ← grayscale masks (same filenames)
│   ├── valImages/
│   │   ├── images/
│   │   └── masks/
│   └── testImages/
│       └── images/          ← test photos for inference
└── outputs/
    └── visualizations/      ← results saved here
```

NOTE Each mask must be a single-channel (grayscale) image where every pixel's value equals the class index (0–9) it belongs to. Pixel value 255 is reserved for unlabeled/void areas and is ignored during training.

Class Definitions & Color Map

The model is trained to detect 10 classes. Each class is assigned a unique BGR color for visualization:

ID	Class Name	Color Label	BGR Color
0	Trees	Forest green	`(34, 139, 34)`
1	Lush Bushes	Lime green	`(50, 205, 50)`
2	Dry Grass	Yellow-green	`(180, 200, 60)`
3	Dry Bushes	Brown-purple	`(42, 42, 165)`
4	Ground Clutter	Gray	`(90, 90, 90)`
5	Flowers	Hot pink	`(255, 0, 200)`
6	Logs	Saddle brown	`(19, 69, 139)`
7	Rocks	Slate	`(180, 130, 70)`
8	Landscape	Tan	`(140, 180, 210)`
9	Sky	Sky blue	`(235, 206, 135)`

How the AI Was Trained — Step by Step

Step 1 — Dataset Preparation

The SegmentationDataset class loads paired image and mask files from disk. Images are opened as RGB and masks as grayscale (L mode). The dataset sorts filenames alphabetically to ensure consistent ordering and filters for valid image extensions (.jpg, .jpeg, .png).

Step 2 — Data Augmentation & Transforms

Before feeding data to the model, two separate transform pipelines are applied:

Training transforms (with augmentation):

- Resize to 512 × 512 pixels
- Random horizontal flip — applied identically to both image and mask so they stay spatially aligned
- Color jitter — randomly varies brightness (±30%), contrast (±30%), and saturation (±20%) to simulate different lighting conditions
- ToTensor — converts PIL image to float tensor in [0, 1] range
- ImageNet normalization — subtracts mean [0.485, 0.456, 0.406] and divides by std [0.229, 0.224, 0.225], matching the pretrained backbone's expected input

Validation/inference transforms (no augmentation):

- Resize → ToTensor → Normalize only (no random flipping or color changes)

Step 3 — Model Architecture

The pipeline first attempts to load LRASPP MobileNetV3-Large from torchvision — a lightweight, fast model well-suited for resource-constrained environments. If unavailable, it falls back to DeepLabV3 with a ResNet-50 backbone.

In both cases, the final convolutional layer is replaced with a fresh Conv2d layer that outputs 10 channels (one per class) instead of the original 21 ImageNet segmentation classes. This technique is called transfer learning — the backbone's pretrained feature extraction is reused while only the output head is retrained.

Step 4 — Loss Function

Pixel-wise Cross-Entropy Loss is used. For each pixel, the model outputs a probability distribution across 10 classes. The loss measures how far the predicted distribution is from the ground truth class label. Because every pixel is classified independently, the total loss is the average over all pixels in the batch. Pixels with mask value 255 are excluded (ignore_index=255).

Step 5 — Optimizer & Learning Rate Scheduler

- AdamW optimizer with learning rate 3×10^{-4} and weight decay 1×10^{-4} (L2 regularization to reduce overfitting)
- Cosine Annealing LR Scheduler — smoothly decays the learning rate from its initial value down to near-zero following a cosine curve over 25 epochs. This helps the model converge more precisely in later epochs without oscillating.
- Gradient clipping at norm 1.0 — prevents exploding gradients that can destabilize training, especially important for pixel-dense outputs

Step 6 — Training Loop (25 Epochs)

Each epoch executes the following steps:

1. A batch of images and masks is loaded and moved to GPU (or CPU if unavailable)
2. Images pass through the model → output tensor of shape (batch, 10, H, W)
3. Output is bilinearly upsampled to match the mask's spatial resolution
4. Cross-Entropy Loss is computed between predictions and ground truth masks
5. optimizer.zero_grad() clears accumulated gradients from the previous step
6. loss.backward() computes gradients for all model parameters via backpropagation
7. Gradients are clipped to max norm 1.0
8. optimizer.step() updates model weights in the direction that reduces loss
9. After all training batches, validation loss is computed with no gradient tracking
10. The learning rate scheduler steps forward
11. If validation loss is the lowest seen so far, the model is saved as segmentation_best.pth

After all 25 epochs, a loss curve PNG is saved showing training and validation loss over time.

Step 7 — Inference

The best saved checkpoint is loaded. For each test image:

- Image is resized, normalized, and wrapped in a batch dimension
- Model performs a forward pass with torch.no_grad() (no gradient tracking needed)
- Output logits are upsampled back to the original image resolution using bilinear interpolation
- torch.argmax selects the highest-confidence class for each pixel, producing a 2D prediction map

Step 8 — Output Visualization

The final composite output image has four panels arranged in a grid:

- Top-left — the original input photo
- Top-right — the colorized segmentation map (each class shown in its assigned color)
- Bottom (full width) — segmentation map overlaid with pill-shaped labels at each class centroid, showing the class name and pixel coverage percentage
- Right strip — a legend panel listing all detected classes and their percentage coverage
- Header bar — filename and title

Classes covering less than 0.3% of the image are hidden from labels and the legend to avoid clutter. Text color on each label (black or white) is automatically chosen based on the background brightness for maximum readability.

⍰ Training Configuration — Quick Reference

Setting	Value
Model	`LRASPP MobileNetV3-Large`
Fallback Model	`DeepLabV3 ResNet-50`

Input Size	512 × 512 pixels
Number of Classes	10
Epochs	25
Batch Size	4
Optimizer	AdamW
Learning Rate	3e-4
Weight Decay	1e-4
LR Scheduler	Cosine Annealing
Loss Function	CrossEntropyLoss (ignore_index=255)
Gradient Clip	max_norm = 1.0
Augmentation	Random H-Flip + Color Jitter
Framework	PyTorch + torchvision
Best Model File	segmentation_best.pth

How to Run

Train the Model

```
python segmentation_pipeline.py --train
```

Run Inference Only

Requires a trained model file (segmentation_best.pth):

```
python segmentation_pipeline.py --infer
```

Train then Infer in One Command

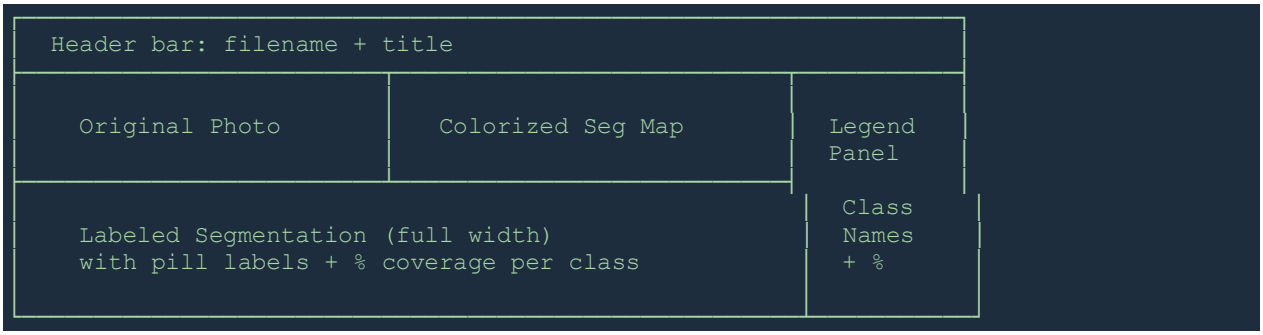
```
python segmentation_pipeline.py --train --infer
```

Use a Custom Model Checkpoint

```
python segmentation_pipeline.py --model my_weights.pth --infer
```

OUTPUT All result images are saved to outputs/visualizations/ with the prefix result_. The loss curve is saved as outputs/visualizations/loss_curve.png.

Output Image Layout



Tips & Troubleshooting

- If training is too slow on CPU, reduce IMG_SIZE to (256, 256) and BATCH_SIZE to 2 in the config
- If you see CUDA out of memory errors, reduce BATCH_SIZE
- Mask pixel values must exactly match class IDs (0–9). Any value outside this range will cause incorrect predictions
- For best results, ensure training data has balanced representation across all 10 classes
- The loss curve (loss_curve.png) is the best indicator of whether training is working — both train and val loss should decrease steadily
- If validation loss stops improving but training loss keeps dropping, the model is overfitting — try adding more augmentation or reducing EPOCHS

Key Files

segmentation_pipeline.py	Main script — contains all training, inference, and visualization logic
segmentation_best.pth	Saved model weights (generated after training)
loss_curve.png	Training vs validation loss chart saved after training
outputs/visualizations/	All result images: original + seg map + labeled output